Дисциплина «Проектирование архитектуры ПО»

Кафедра корпоративных информационных систем

ОТЧЁТ

по практической работе №2

Тема: CRUD-операции с различными базами данных

Выполнил: студент группы БСБО-01-23

Судовский Андрей Леонидович

Москва, 2026

СОДЕРЖАНИЕ

1. Цель работы 3

2. Задание 3

3. Предметная область и набор данных 3

4. Операции CRUD в пяти СУБД 4

4.1. Redis 5

4.2. MongoDB 7

4.3. Neo4j 9

4.4. Elasticsearch 12

4.5. PostgreSQL 14

5. Выводы 16

Список использованных источников 17

Приложение А. Схема журнала посещений (pg_schema.sql) 19

Приложение Б. Маппинг индекса lecture_courses (es_mapping.json) 19

Приложение В. Создание коллекции student_groups (mongo_create.js) 20

1. Цель работы

Освоить базовые операции создания, чтения, изменения и удаления данных (CRUD) в пяти СУБД разных моделей — Redis, MongoDB, Neo4j, Elasticsearch и PostgreSQL — на данных одной предметной области и сравнить, как одни и те же сведения о студентах, группах и курсах представляются в каждой модели.

2. Задание

«Для каждого из типов хранения, развернутых в рамках практической работы №1, выполнить следующие операции: создать хранилище данных; добавить данные в хранилище; прочитать данные из хранилища по ключу; изменить и сохранить данные в хранилище; удалить записи в хранилище; удалить хранилище».

Данные — список студентов группы и изучаемых курсов в рамках семестра: для Redis — только список студентов с ключом в виде номера зачётной книжки; для MongoDB — документ с данными и составом группы; для Neo4j — связи между группой, студентом и курсом с учётом курсов по выбору; для Elasticsearch — данные с полнотекстовым описанием курса; для PostgreSQL — данные о посещении лекций студентами с партиционированием по неделе посещения, сформированные случайным выбором.

Работа выполнена по методическим указаниям [3] к заданию [1] на стенде практической работы №1 [2]. Команды CRUD составлены по документации СУБД [4–8], особенности моделей данных — по [9, 10].

3. Предметная область и набор данных

Работа выполнена на стенде практической работы №1 (рабочий каталог `SAD\practice-db\pr2`). В отличие от примеров методических указаний, построенных на интернет-магазине, использованы данные учебного процесса, которые потребуются в практической работе №3 и лабораторных

работах. Набор формирует генератор `common\dataset.py`; файлы для каждой СУБД готовит `pr2\make_data.py`.

– Две группы направления 38.03.05 «Бизнес-информатика» (кафедра корпоративных информационных систем) набора 2023 года: БСБО-01-23 — 25 студентов, БСБО-02-23 — 6 студентов. Номер зачётной книжки имеет вид `23Б0101`.
– Семь курсов 6-го семестра (весна 2026 года): пять обязательных и два курса по выбору — «NoSQL и полиглотное хранение» и «Бизнес-аналитика». У каждого студента один или оба курса по выбору.
– Расписание лекций: 88 лекций, 176 занятий (по каждой лекции для двух групп), с 9 февраля 2026 года.
– Идентификаторы — UUID версии 5 от естественного ключа (номер зачётки, название курса, группы). Одна и та же сущность получает один и тот же идентификатор во всех хранилищах и при повторной генерации.

Таблица 1 — Размещение данных по СУБД

СУБД	**Что хранится**	**Ключ**	**Структура**
Redis	студенты группы БСБО-01-23	номер зачётной книжки, например `23Б0101`	хеш (Hash) с полями `last_name`, `first_name`, `email`, `status` и др.; логическая база №1
MongoDB	группы с составом	название группы (уникальный индекс), `_id` = UUID группы	документ группы с вложенными объектами специальности, кафедры и массивом студентов
Neo4j	связи группа — студент — курс	`Student.student_card_number`, `Group.name`, `LectureCourse.id`	узлы `Group`, `Student`, `LectureCourse`; связи `MEMBER_OF`, `STUDIES`, `CHOSE`
Elasticsearch	курсы с описанием	`_id` = UUID курса	документ курса в индексе `lecture_courses` с русским анализатором
PostgreSQL	посещаемость лекций	первичный ключ (`id`, `week_start_date`)	таблица `attendance`, партиционированная по понедельнику недели, + справочники `students`, `schedule`

4. Операции CRUD в пяти СУБД

Все операции собраны в сценарий `run_all.sh`. Он выполняет шесть шагов задания для выбранной СУБД (`./run_all.sh redis` …

./run_all.sh pg) и сохраняет вывод в каталог pr2\evidence. Ключ --keep пропускает последний шаг, чтобы данные остались для просмотра в веб-интерфейсах. Соответствие шагов задания и команд — в таблице 2.

Таблица 2 — Операции задания и их реализация

Операция	Redis	MongoDB	Neo4j	Elasticsearch	PostgreSQL
Создать хранилище	логическая база -n 1	createCollection с $jsonSchema, createIndex	CREATE CONSTRAINT	PUT /lecture_courses (маппинг)	CREATE DATABASE, CREATE TABLE … PARTITION BY RANGE
Добавить данные	HSET (через --pipe)	insertMany	UNWIND … MERGE	_bulk	INSERT … SELECT random()
Прочитать по ключу	HGETALL 23Б0101	findOne({name})	MATCH (s {student_card_number})	GET _doc/<id>, _search	SELECT … WHERE student, week
Изменить	HSET существующих полей	$set, $push, arrayFilters	DELETE r … MERGE	POST _update/<id>	UPDATE … RETURNING
Удалить записи	DEL	$pull, deleteOne	DETACH DELETE	DELETE _doc/<id>	DELETE, DETACH PARTITION
Удалить хранилище	FLUSHDB	drop(), dropDatabase()	DETACH DELETE всех узлов, DROP CONSTRAINT	DELETE /lecture_courses	DROP TABLE, DROP DATABASE

4.1. Redis

Хранилищем служит логическая база Redis №1 (параметр -n 1): она отделена от данных практической работы №1 в базе №0 и удаляется целиком командой FLUSHDB, не затрагивая другие базы. Перед загрузкой база пуста (DBSIZE = 0). Студенты добавляются 25 командами HSET в формате протокола Redis через redis-cli --pipe (ответ errors: 0, replies: 25). Ключ каждого хеша — номер зачётной книжки, поэтому чтение по ключу выполняется одной командой HGETALL 23Б0101 (рисунок 1). Загруженные ключи видны в Redis Commander, подключённом к базе №1 (рисунок 2).

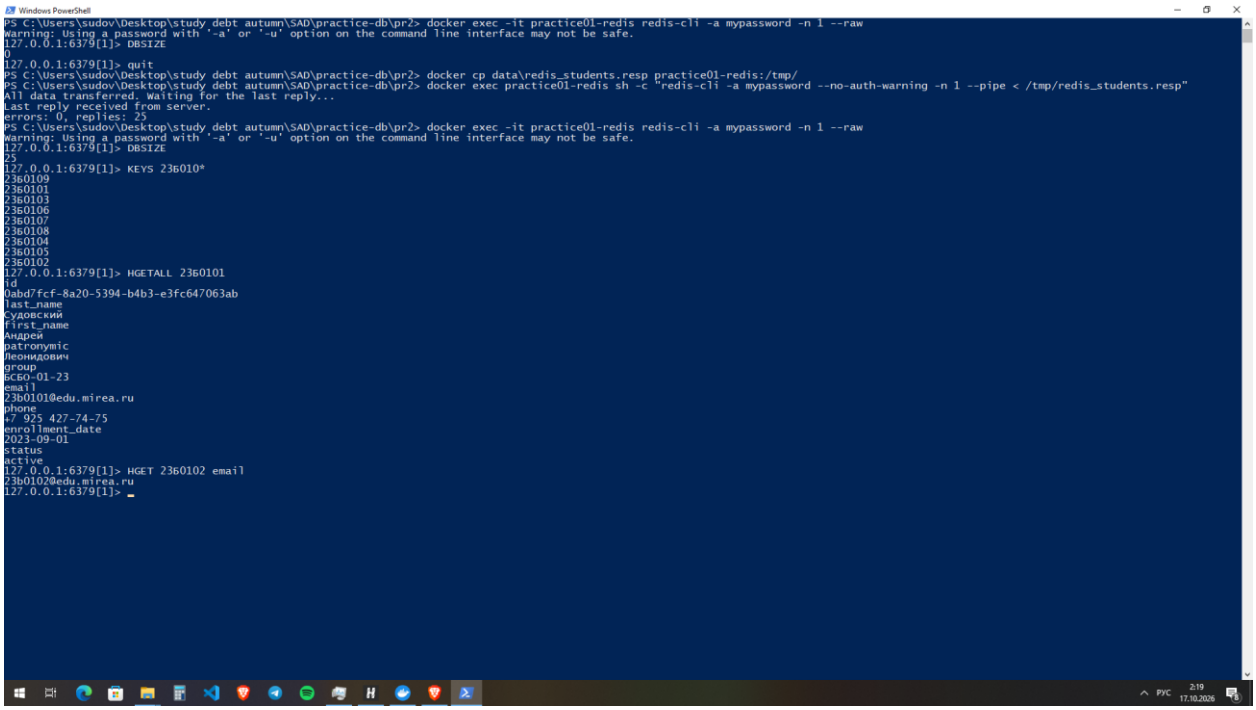


Рисунок 1 — Redis: создание хранилища, загрузка студентов и чтение по ключу

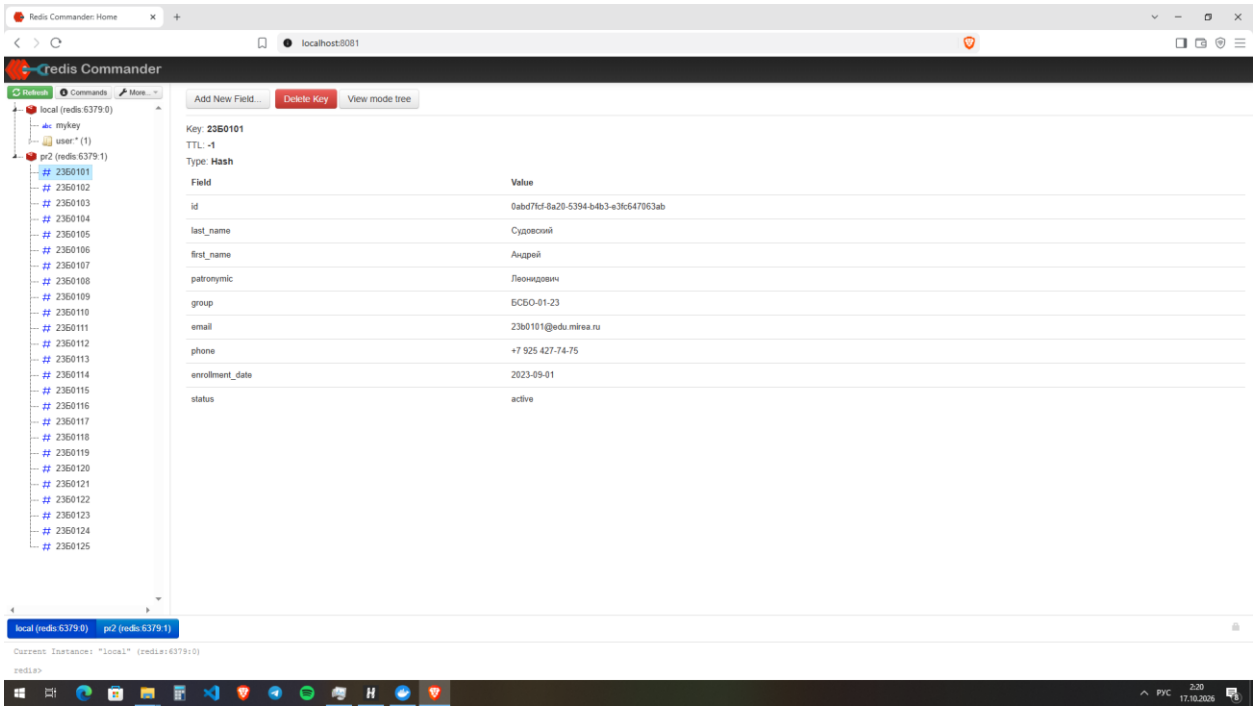


Рисунок 2 — Redis Commander: студенты в базе №1, хеш 23Б0101

`HSET` по существующим полям изменяет значения и возвращает 0 — число новых полей; результат проверен `HMGET`. Запись удаляется `DEL` (после неё `EXISTS` = 0, ключей 24), хранилище — `FLUSHDB` (рисунок 3).

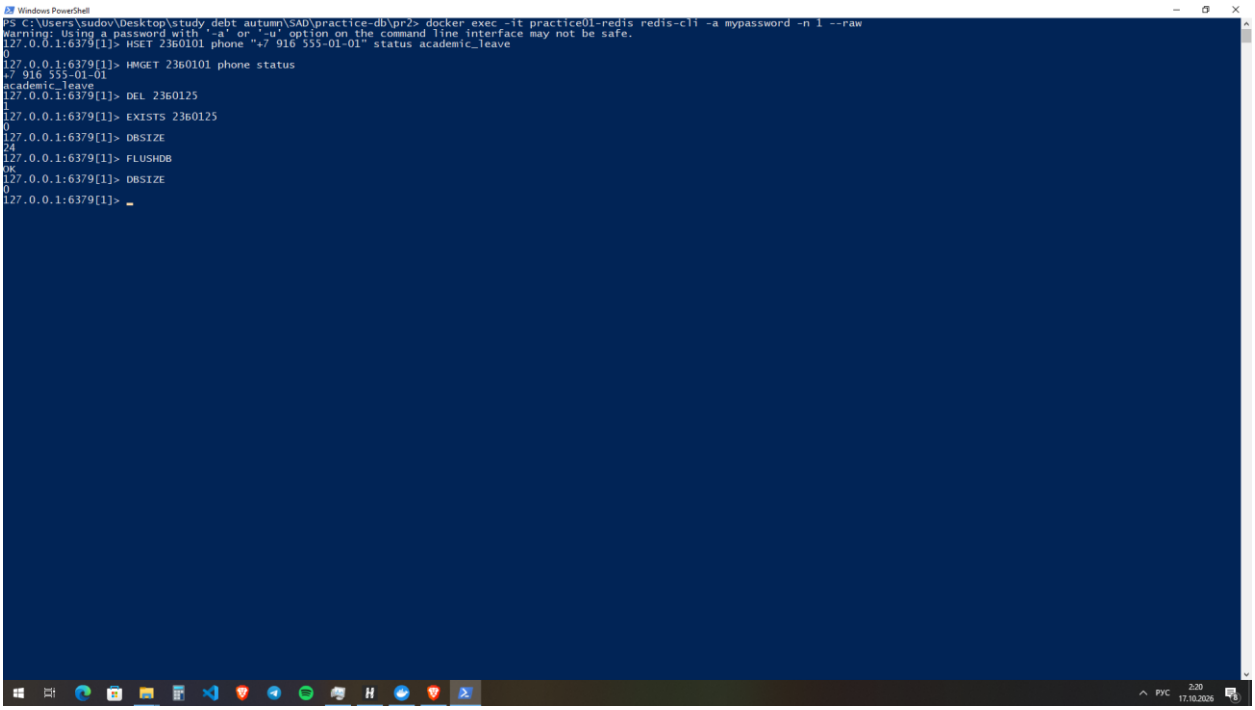


Рисунок 3 — Redis: изменение, удаление записи и удаление хранилища

4.2. MongoDB

Хранилище — база `university` и коллекция `student_groups`. Коллекция создаётся явно, с правилом проверки `$jsonSchema`: обязательны поля `name`, `enrollment_year`, `specialty`, `students`, а название группы должно соответствовать шаблону вида «БСБО-01-23». Уникальный индекс по `name` делает название группы ключом. Два документа групп добавлены `insertMany`. Чтение по ключу `findOne({name: "БСБО-01-23"})` возвращает группу целиком — специальность, кафедру и состав — одним обращением (рисунок 4; массив студентов урезан проекцией `$slice`). Документ в Mongo Express — рисунок 5.

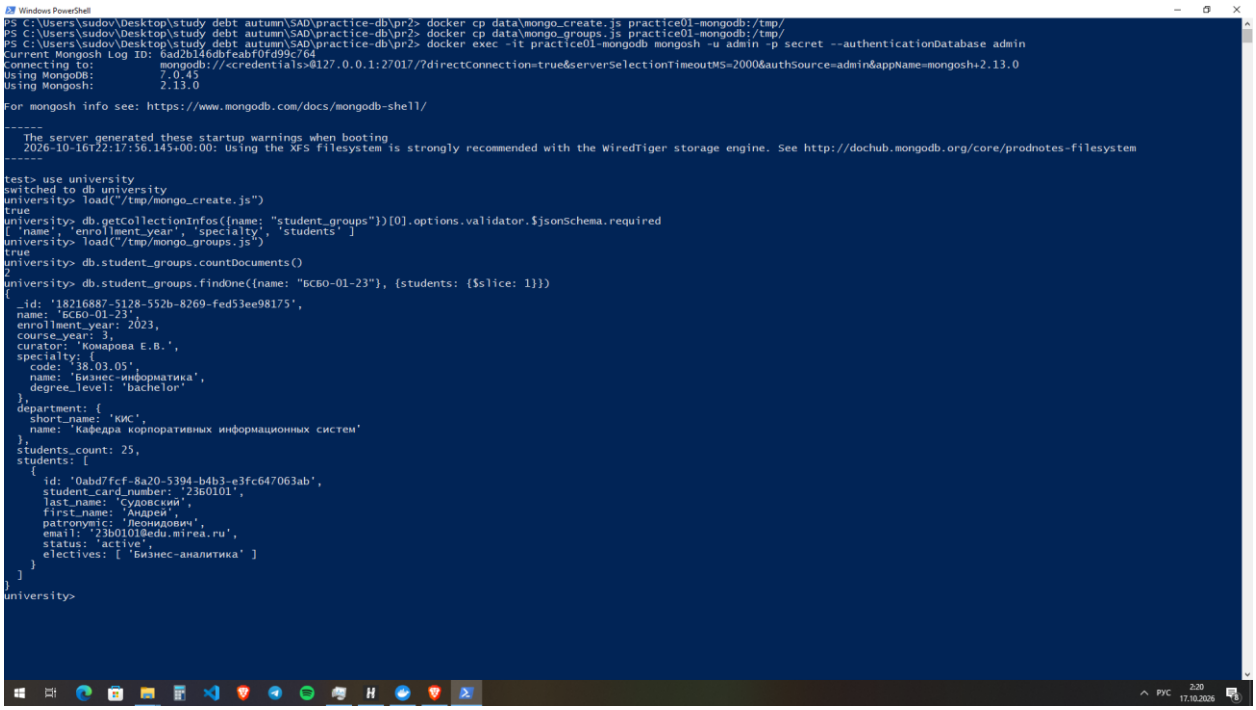


Рисунок 4 — MongoDB: создание коллекции с проверкой схемы, добавление и чтение документа группы

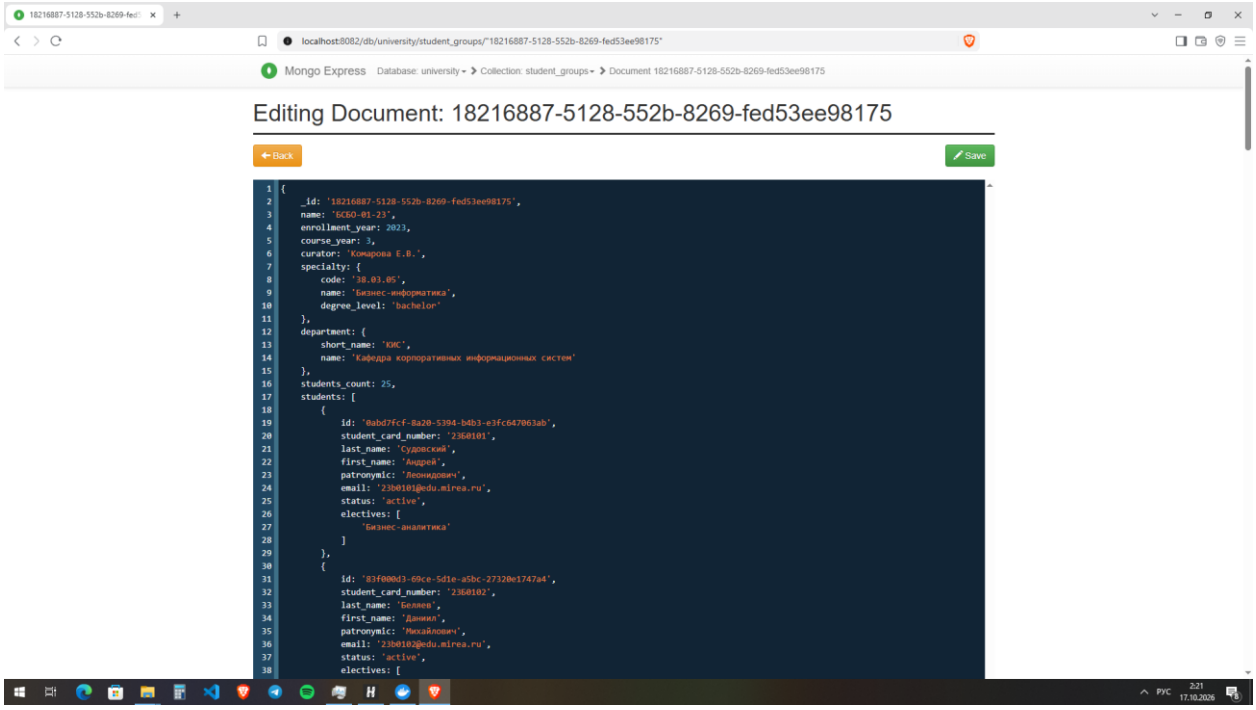


Рисунок 5 — Mongo Express: документ группы БСБО-01-23 с вложенным составом

Изменения выполнены операторами обновления без перезаписи документа (рисунок 6): `$set` меняет куратора, `$push` вместе с `$inc` добавляет студента в состав и увеличивает счётчик, `$set` с `arrayFilters` меняет статус конкретного студента внутри массива. Удаление записей — `$pull` (студент из состава) и `deleteOne` (документ группы БСБО-02-23);

удаление хранилища — `drop()` и `dropDatabase()`, после чего база `university` отсутствует в списке `show dbs` (рисунок 7).

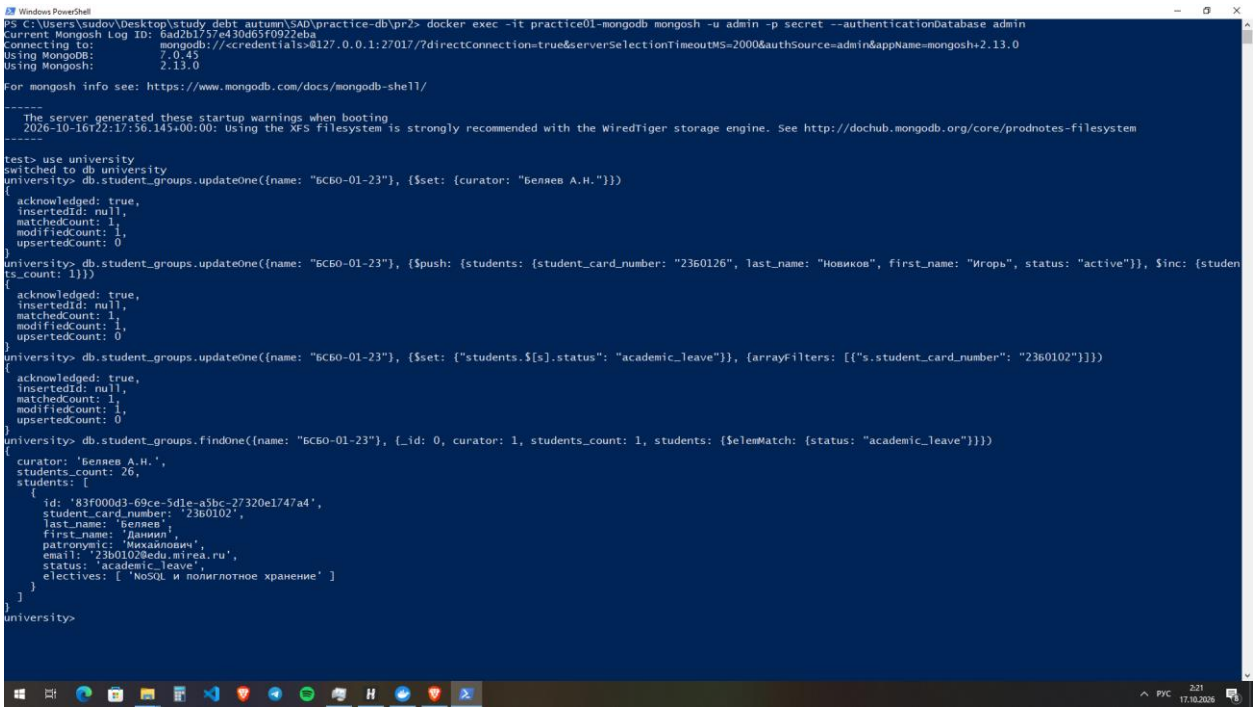


Рисунок 6 — MongoDB: изменение документа группы ($set, $push, arrayFilters)

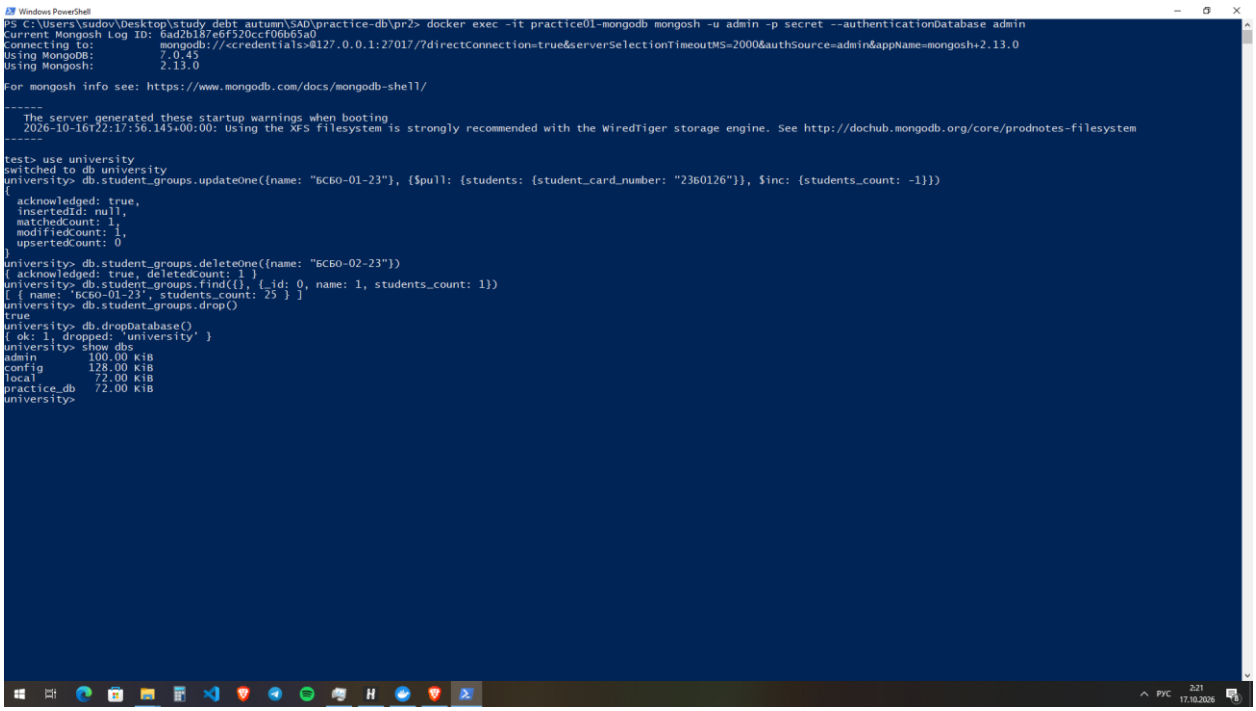


Рисунок 7 — MongoDB: удаление записей и удаление хранилища

4.3. Neo4j

Модель графа: `(:Student)-[:MEMBER_OF]->(:Group)`, `(:Group)-[:STUDIES]->(:LectureCourse)` — обязательные курсы изучает вся

группа, `(:Student)-[:CHOSE]->(:LectureCourse)` — курс по выбору выбирает каждый студент индивидуально. Так учтены курсы по выбору: состав курсов студента — это объединение курсов его группы и его собственных выборов. Хранилище создаётся ограничениями уникальности по ключам узлов (они же создают индексы); загрузка выполняется `UNWIND … MERGE` и даёт 2 группы, 7 курсов, 31 студента и 80 связей (рисунок 8). Фрагмент графа — рисунок 9.

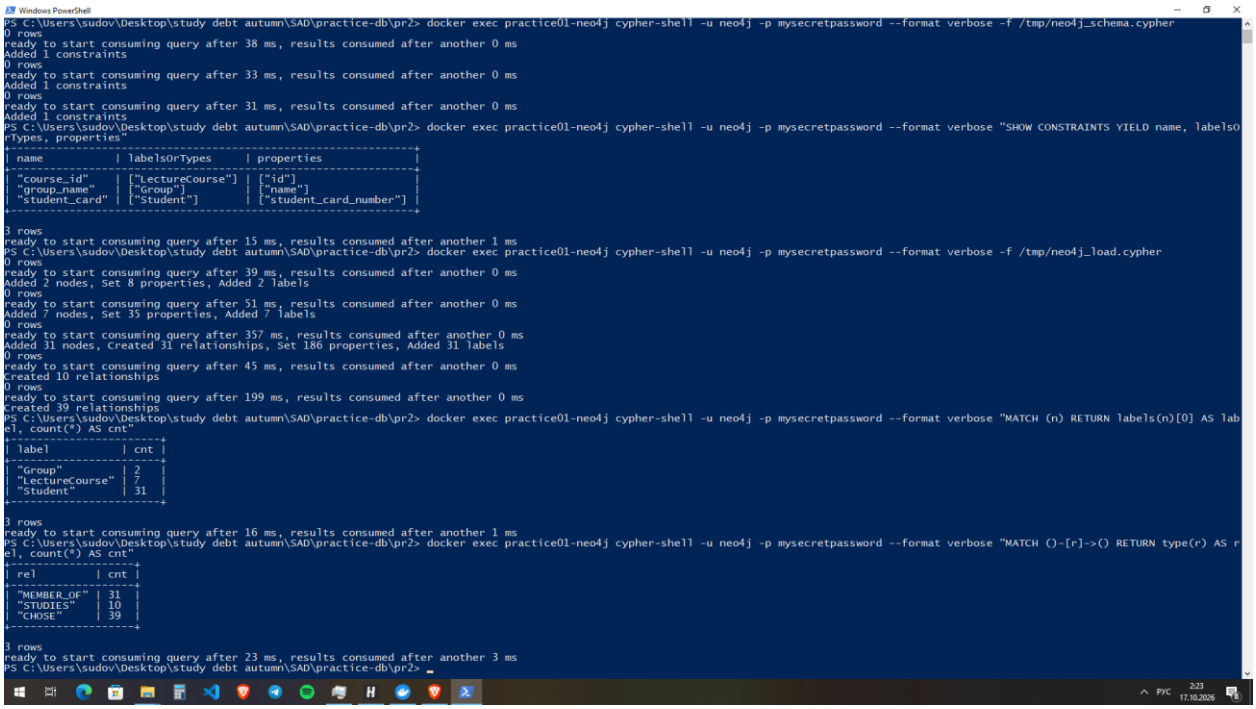


Рисунок 8 — Neo4j: ограничения уникальности и загрузка узлов и связей

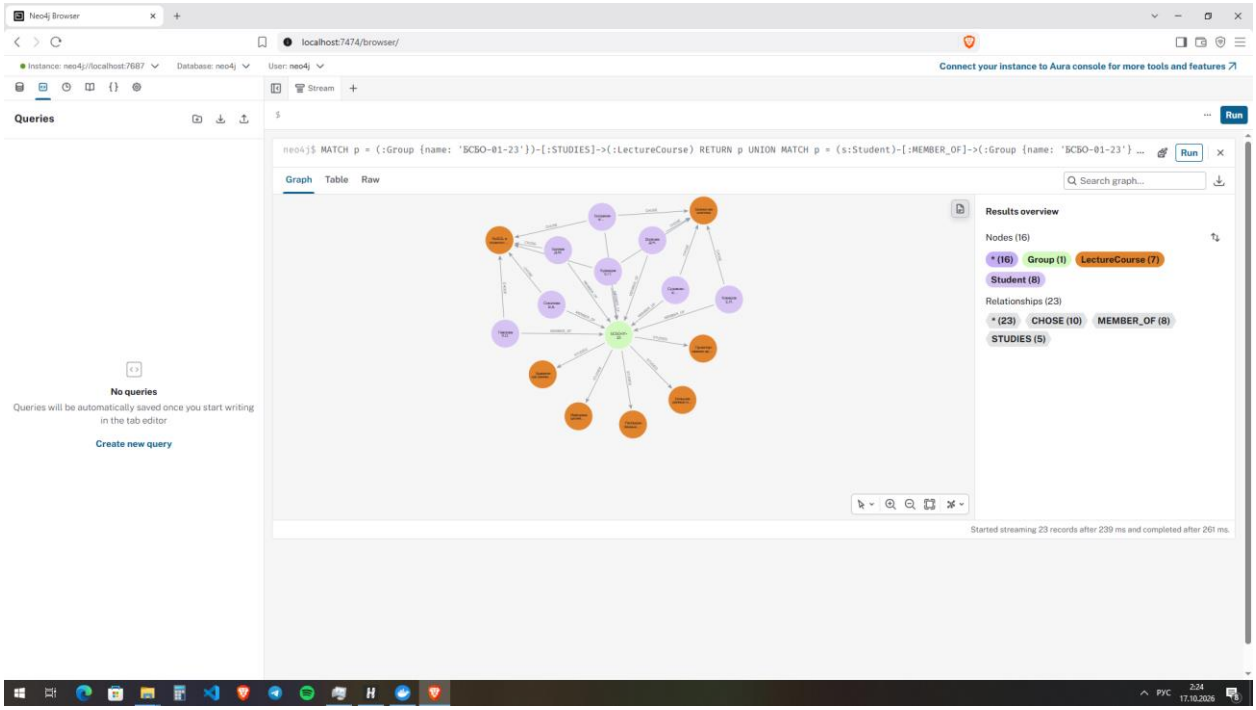


Рисунок 9 — Neo4j Browser: группа, её обязательные курсы и выбор курсов студентами

Чтение по ключу — все курсы студента `23Б0101` запросом `UNION` по двум путям графа. Изменение — замена курса по выбору: удаление связи `CHOSE` и создание новой (`MERGE`). Удаление записи — `DETACH DELETE` студента вместе с его связями (обычный `DELETE` узла со связями завершился бы ошибкой). Удаление хранилища — удаление всех узлов трёх меток и ограничений (рисунок 10).

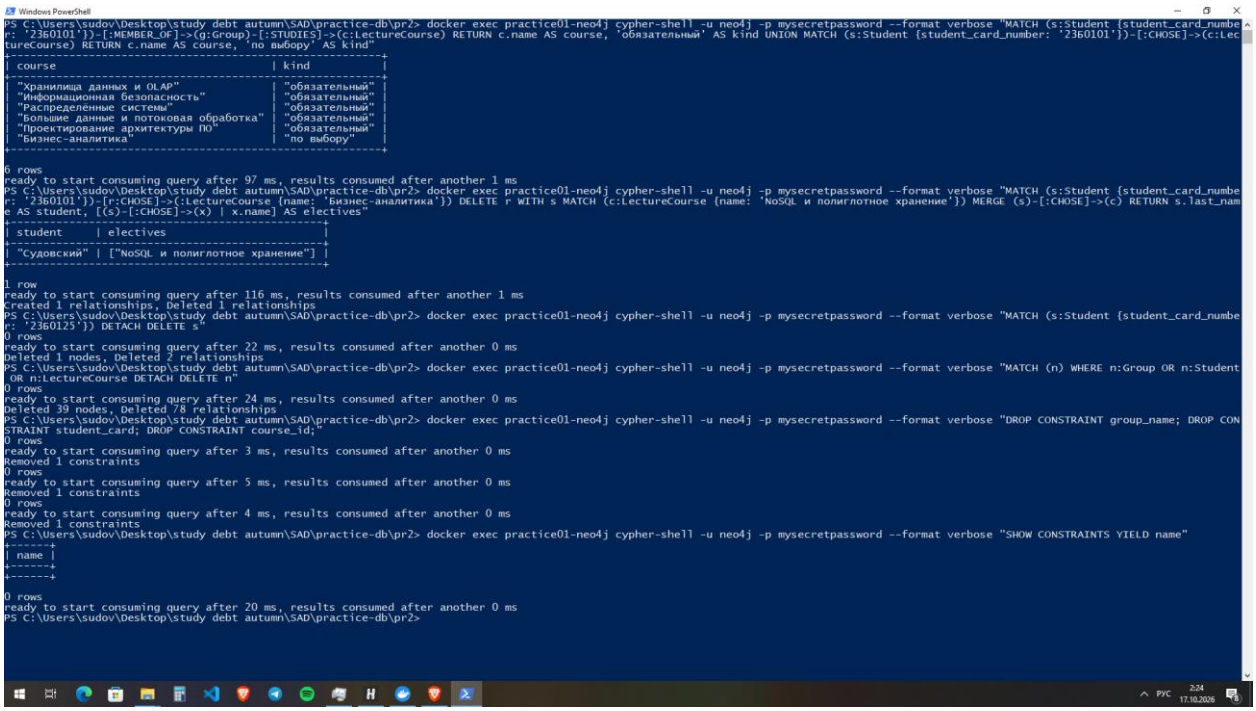


Рисунок 10 — Neo4j: чтение курсов студента, изменение, удаление записи и хранилища

4.4. Elasticsearch

Хранилище — индекс `lecture_courses` с явным маппингом и пользовательским анализатором `russian_custom` (токенизатор `standard`, фильтры `lowercase`, русские стоп-слова и стемминг), как в методических указаниях к практической работе №3. Описание курса — поле `text` с этим анализатором, название — `text` с подполем `keyword` для точной фильтрации, теги и код специальности — `keyword`. Курсы загружены через Bulk API (7 документов), чтение по ключу — `GET _doc/<UUID курса>` (рисунок 11).

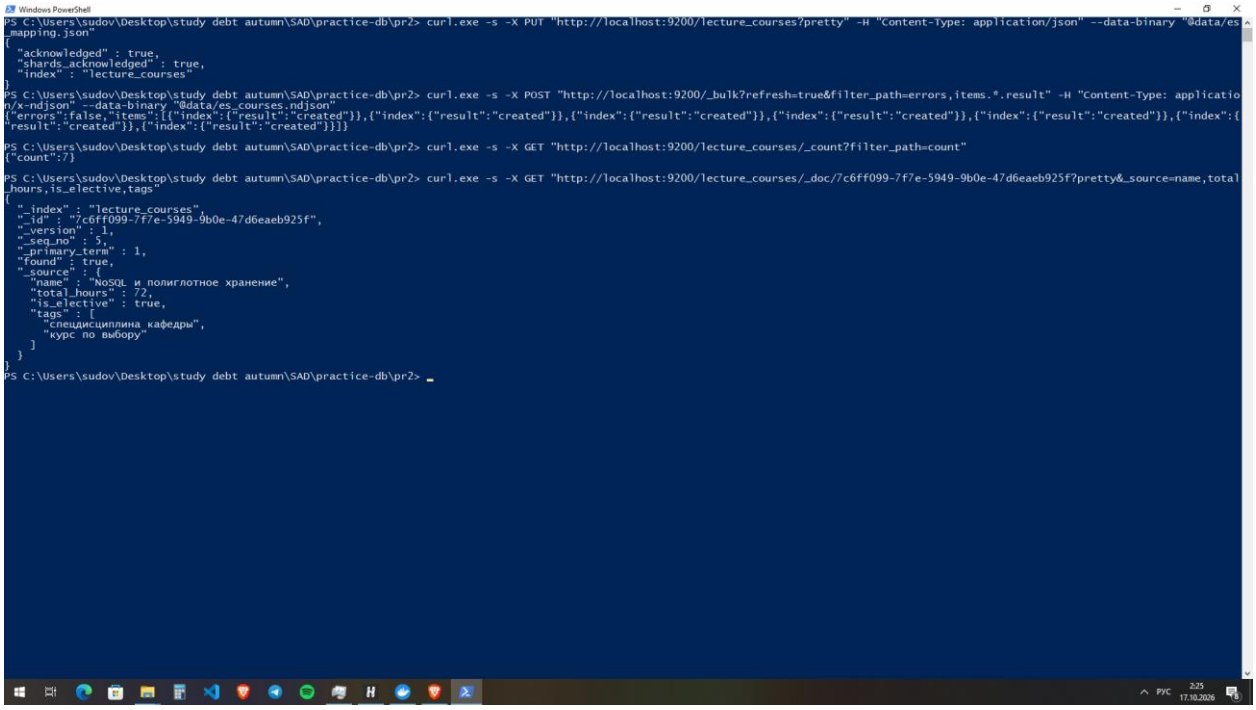


Рисунок 11 — Elasticsearch: создание индекса с маппингом, загрузка и чтение документа по ключу

Полнотекстовый поиск по описанию учитывает морфологию: запрос «партиционированием таблиц» находит курс, в описании которого написано «партиционирование таблиц», и подсвечивает совпадения (рисунок 12).

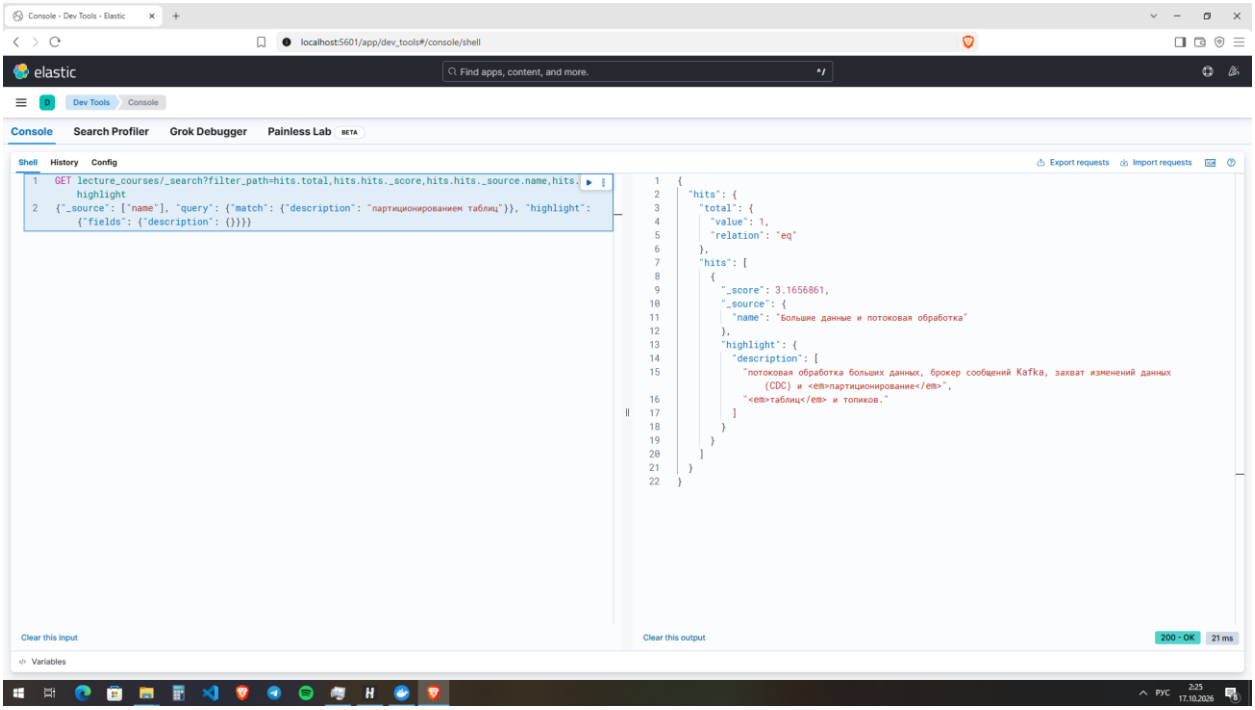


Рисунок 12 — Kibana Dev Tools: полнотекстовый поиск по описанию курса с учётом морфологии

Частичное обновление `_update` меняет поля документа и увеличивает номер версии (`_version` = 2); документ удаляется `DELETE _doc/<id>` (в индексе остаётся 6 документов), индекс — `DELETE /lecture_courses` (рисунок 13).

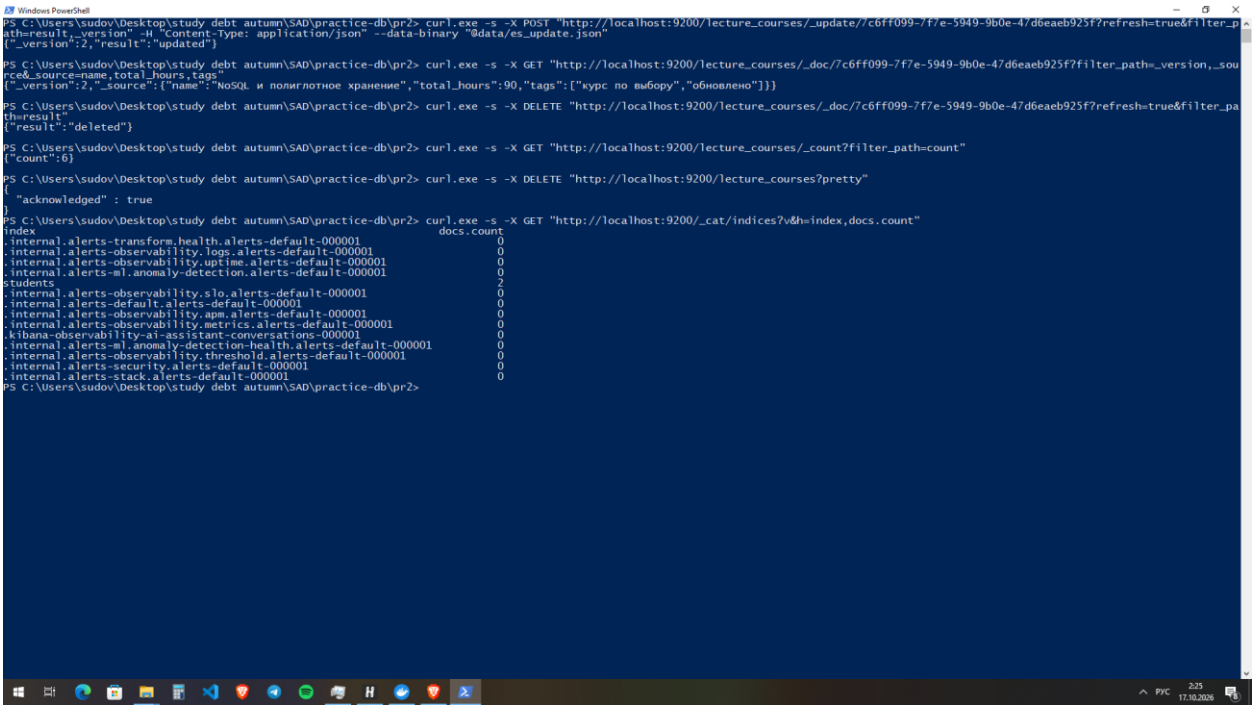


Рисунок 13 — Elasticsearch: изменение и удаление документа, удаление индекса

4.5. PostgreSQL

Хранилище — отдельная база `pr2_attendance`. Таблица `attendance` объявлена как секционированная `PARTITION BY RANGE (week_start_date)`, где `week_start_date` — понедельник недели занятия. Первичный ключ включает ключ партиционирования (`id`, `week_start_date`) — это требование PostgreSQL к секционированным таблицам. Партиции создаёт функция `create_weekly_partitions` из методических указаний: 20 недель весеннего семестра (`attendance_w2026_02_09` … `attendance_w2026_06_22`) и партиция по умолчанию для дат вне семестра (рисунок 14, листинг — приложение А).

Отметки посещаемости сформированы случайным выбором средствами SQL: каждому студенту назначена своя вероятность посещения от 0,55 до 0,98, отметка равна `random() < вероятность`, а `setseed` делает выборку воспроизводимой. Для каждого занятия группы отметка ставится только тем студентам, которые слушают курс (обязательный или выбранный). Получено 2544 отметки в 16 недельных партициях; доля посещений по неделям — от 69 до 80 % (рисунок 15).

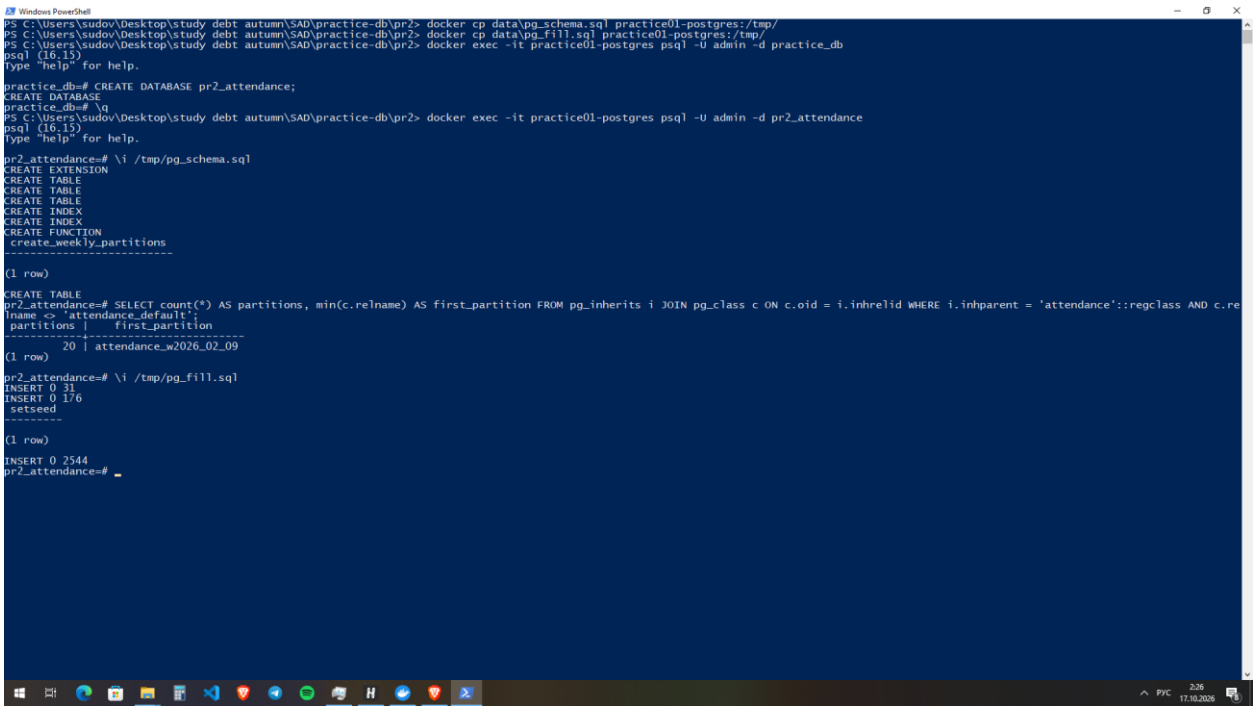


Рисунок 14 — PostgreSQL: создание базы и секционированной таблицы, добавление данных

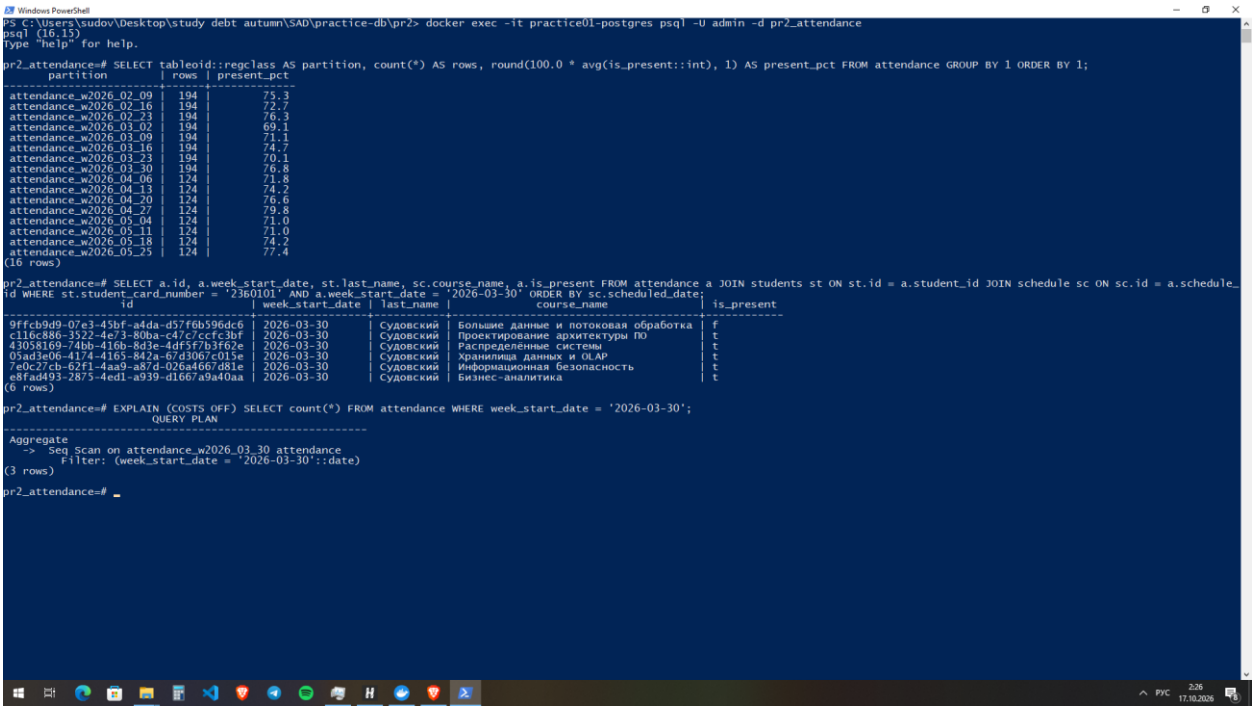


Рисунок 15 — PostgreSQL: распределение строк по партициям, чтение по ключу, отсечение партиций

Чтение по ключу — отметки студента за неделю 30 марта 2026 года. План запроса с условием на `week_start_date` показывает отсечение партиций (partition pruning): сканируется только партиция `attendance_w2026_03_30`, а не вся таблица. Партиции и данные таблицы в pgAdmin — рисунок 16.

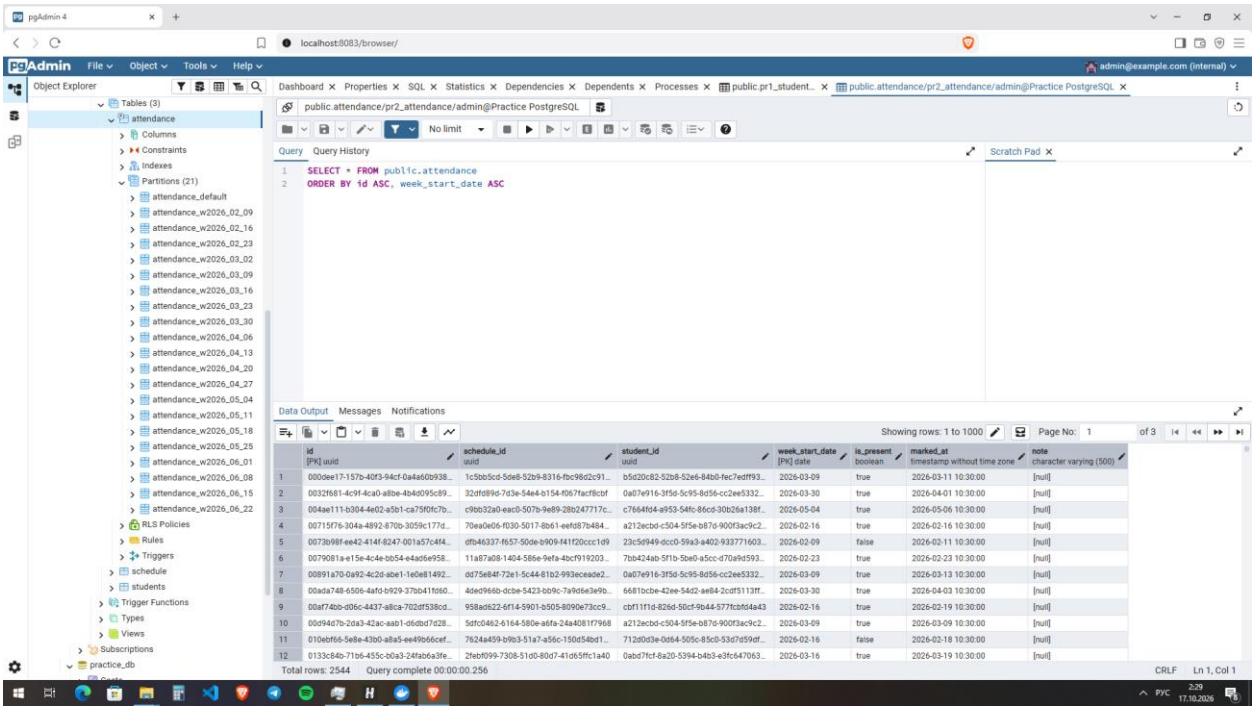


Рисунок 16 — pgAdmin 4: партиции таблицы attendance и её данные

Изменение — исправление первого пропуска студента с пометкой «исправлено по справке» (`UPDATE … RETURNING`). Удаление записей выполнено двумя способами: `DELETE` отметок одного студента (80 строк) и отсоединение и удаление целой недельной партиции (`DETACH PARTITION` + `DROP TABLE`) — так за одну операцию удаляются данные недели без построчного `DELETE`. Хранилище удалено командами `DROP TABLE attendance` и `DROP DATABASE pr2_attendance WITH (FORCE)`: параметр `FORCE` (PostgreSQL 13+) закрывает оставшиеся подключения к базе, в данном случае — сеанс pgAdmin (рисунок 17).

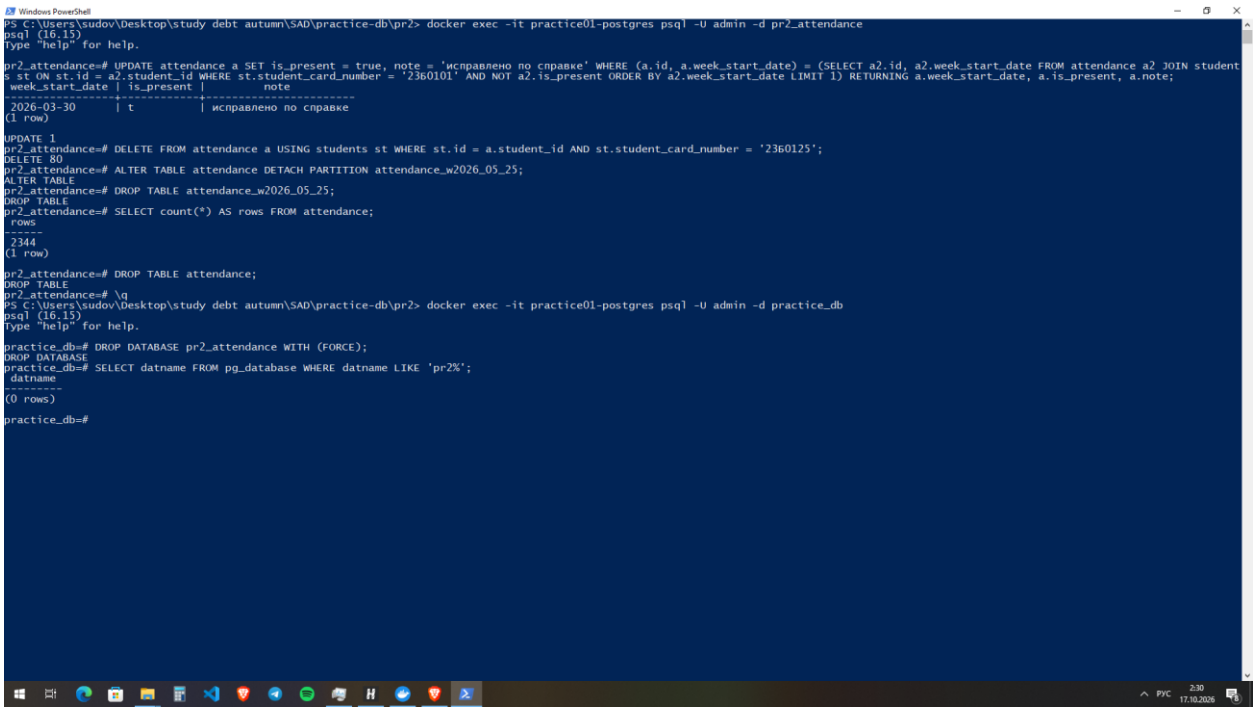


Рисунок 17 — PostgreSQL: изменение, удаление записей и партиции, удаление хранилища

5. Выводы

Во всех пяти СУБД выполнены шесть операций задания: создание хранилища, добавление данных, чтение по ключу, изменение, удаление записей и удаление хранилища. Данные одной предметной области — студенты, группы, курсы и посещаемость — представлены в каждой модели так, как ей свойственно: в Redis — хеш на студента с ключом-номером зачётки, в MongoDB — агрегат «группа с составом», в Neo4j — граф связей, где курсы по выбору выражены индивидуальными связями студентов, в

Elasticsearch — документы курсов с полнотекстовым описанием и морфологическим поиском, в PostgreSQL — журнал посещений с недельными партициями.

Сравнение показало, что понятие «хранилище» в разных СУБД означает разное: логическую базу Redis, коллекцию MongoDB, набор ограничений и меток в Neo4j, индекс Elasticsearch, базу и таблицу PostgreSQL. Схема строгая в PostgreSQL и задаётся маппингом в Elasticsearch, а в MongoDB и Neo4j она гибкая и ограничивается по желанию (`$jsonSchema`, ограничения уникальности). Партиционирование журнала посещений по неделям позволяет читать данные одной недели из одной партиции и удалять неделю целиком операцией над партицией.

Работа выполнена 17.10.2026; рисунки сняты в тот же день с экрана рабочего компьютера. Сценарий `run_all.sh` воспроизводит все операции, данные генерируются детерминированно.

Список использованных источников

1. Задание на практические и лабораторные работы по дисциплине «Проектирование архитектуры ПО». — Москва, 2026. — Текст : электронный.

2. Практическая работа №1. Развертывание контейнеризированных сред для нескольких типов баз данных : методические указания. — 2026. — 43 с. — Текст : электронный.

3. Практическая работа №2. CRUD-операции с различными базами данных : методические указания. — 2026. — 71 с. — Текст : электронный.

4. Redis Documentation. — URL: https://redis.io/docs/latest/ (дата обращения: 17.10.2026). — Текст : электронный.

5. MongoDB Manual. — URL: https://www.mongodb.com/docs/manual/ (дата обращения: 17.10.2026). — Текст : электронный.

6. Neo4j Cypher Manual. — URL: https://neo4j.com/docs/cypher-manual/current/ (дата обращения: 17.10.2026). — Текст : электронный.

7. Elasticsearch Guide 8.16. — URL: https://www.elastic.co/guide/en/elasticsearch/reference/8.16/index.html (дата обращения: 17.10.2026). — Текст : электронный.

8. PostgreSQL 16 Documentation. — URL: https://www.postgresql.org/docs/16/index.html (дата обращения: 17.10.2026). — Текст : электронный.

9. Садаладж П. Дж., Фаулер М. NoSQL: новая методология разработки нереляционных баз данных / П. Дж. Садаладж, М. Фаулер. — Москва : Вильямс, 2013. — 192 с. — Текст : непосредственный.

10. Клеппман М. Высоконагруженные приложения. Программирование, масштабирование, поддержка / М. Клеппман. — Санкт-Петербург : Питер, 2018. — 640 с. — Текст : непосредственный.

Приложение А. Схема журнала посещений (pg_schema.sql)

```
-- Справочник студентов и расписание (минимальный набор для журнала посещений)
CREATE EXTENSION IF NOT EXISTS "uuid-ossp";


CREATE TABLE students (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    student_card_number VARCHAR(20) NOT NULL UNIQUE,
    last_name VARCHAR(100) NOT NULL,
    first_name VARCHAR(100) NOT NULL,
    patronymic VARCHAR(100),
    group_name VARCHAR(50) NOT NULL
);


CREATE TABLE schedule (
    id UUID PRIMARY KEY DEFAULT uuid_generate_v4(),
    group_name VARCHAR(50) NOT NULL,
    course_name VARCHAR(500) NOT NULL,
    lecture_number INT NOT NULL,
    scheduled_date DATE NOT NULL,
    week_start_date DATE NOT NULL,       -- понедельник недели = ключ партиционирования журнала
    classroom VARCHAR(50),
    teacher_name VARCHAR(300)
);


-- Журнал посещений лекций, партиционированный по неделе посещения (RANGE по понедельнику
    недели)
CREATE TABLE attendance (
    id UUID NOT NULL DEFAULT uuid_generate_v4(),
    schedule_id UUID NOT NULL,
    student_id UUID NOT NULL,
    week_start_date DATE NOT NULL,
    is_present BOOLEAN NOT NULL DEFAULT FALSE,
    marked_at TIMESTAMP DEFAULT NOW(),
    note VARCHAR(500),
    PRIMARY KEY (id, week_start_date)    -- ключ партиционирования обязан входить в первичный
        ключ
) PARTITION BY RANGE (week_start_date);


CREATE UNIQUE INDEX uq_attendance_schedule_student ON attendance (schedule_id, student_id,
    week_start_date);
CREATE INDEX idx_attendance_student ON attendance (student_id);


-- Партиции на N недель вперёд: attendance_wГГГГ_ММ_ДД = [понедельник, следующий понедельник)
CREATE OR REPLACE FUNCTION create_weekly_partitions(start_date DATE, num_weeks INT) RETURNS
    VOID AS $$
DECLARE
    week_start DATE := date_trunc('week', start_date)::DATE;
    part TEXT;
BEGIN
    FOR i IN 0..num_weeks - 1 LOOP
        part := 'attendance_w' || to_char(week_start, 'YYYY_MM_DD');
        IF NOT EXISTS (SELECT 1 FROM pg_class WHERE relname = part) THEN
            EXECUTE format('CREATE TABLE %I PARTITION OF attendance FOR VALUES FROM (%L) TO
                (%L)',
                           part, week_start, week_start + 7);
        END IF;
        week_start := week_start + 7;
    END LOOP;
END;
$$ LANGUAGE plpgsql;


SELECT create_weekly_partitions('2026-02-09', 20);          -- весенний семестр 2026: 20 недель
CREATE TABLE attendance_default PARTITION OF attendance DEFAULT;   -- даты вне семестра
```

Приложение Б. Маппинг индекса lecture_courses (es_mapping.json)

```
{
  "settings": {
    "number_of_shards": 1,
    "number_of_replicas": 0,
    "analysis": {
```

```
      "analyzer": {
        "russian_custom": {
          "type": "custom",
          "tokenizer": "standard",
          "filter": [
            "lowercase",
            "russian_stop",
            "russian_stemmer"
          ]
        }
      },
      "filter": {
        "russian_stop": {
          "type": "stop",
          "stopwords": "_russian_"
        },
        "russian_stemmer": {
          "type": "stemmer",
          "language": "russian"
        }
      }
    }
  },
  "mappings": {
    "properties": {
      "course_id": {
        "type": "keyword"
      },
      "name": {
        "type": "text",
        "analyzer": "russian_custom",
        "fields": {
          "keyword": {
            "type": "keyword"
          }
        }
      },
      "description": {
        "type": "text",
        "analyzer": "russian_custom"
      },
      "specialty_code": {
        "type": "keyword"
      },
      "department": {
        "type": "keyword"
      },
      "semester": {
        "type": "integer"
      },
      "total_hours": {
        "type": "integer"
      },
      "is_elective": {
        "type": "boolean"
      },
      "tags": {
        "type": "keyword"
      },
      "terms": {
        "type": "keyword"
      }
    }
  }
}
```

Приложение В. Создание коллекции student_groups (mongo_create.js)

```
// Создание хранилища: коллекция с проверкой структуры документа и уникальным индексом по
    названию группы
db.createCollection("student_groups", {
  validator: { $jsonSchema: {
    bsonType: "object",
    required: ["name", "enrollment_year", "specialty", "students"],
    properties: {
      name:            { bsonType: "string", pattern: "^[А-ЯЁ]{4}-[0-9]{2}-[0-9]{2}$" },
      enrollment_year: { bsonType: "number", minimum: 2000 },
      students:        { bsonType: "array", items: { bsonType: "object",
```

```
                        required: ["student_card_number", "last_name", "first_name"] } }
    }
  } }
});
db.student_groups.createIndex({ name: 1 }, { unique: true });
```